Evolving Super Mario Bros Controllers via Typed Genetic Programming

Miguel Cabral Pinto and Tiago Silva

Departamento de Engenharia Informática, Universidade de Coimbra
{mcp, tds}@student.uc.pt

Abstract. We evolve game controllers for Super Mario Bros using Genetic Programming across three task variants: a Runner targeting level completion, a Hunter targeting enemy kills, and a Thief targeting coin collection. Each individual is a typed syntax tree that compiles to an executable Python function, so the search operates directly over the space of structured programs. Bloat is controlled through Double Tournament selection with parsimony pressure; generalization is promoted through a rotating seed pool that changes every 10 generations. Runner agents achieve the highest win rate, while both Hunter and Thief agents notably improve on kill/coin collection. The Random baseline fails to create capable agents.

1 Introduction

In this work we use Genetic Programming (GP) [2] to evolve controllers for the Mario AI framework [4]. GP is well-matched to this problem for two reasons: it requires only a scalar fitness signal and no gradient information, and it produces programs as output, meaning the evolved controller is a readable Python function. Since the failure mode of any agent is visible in the code, iterating on the fitness function is fast and targeted.

The course provided a base GP repository implemented with DEAP [1]: a minimal random search over a small typed grammar with two senses (`on_ground`, `can_jump`), two actions (`RIGHT`, `JUMP`), and basic `IF`/`SEQ` program structure. During our work, we expanded the grammar to give agents better perception of terrain and enemies, designed the fitness function, and replaced the random search with a full evolutionary algorithm.

Three task variants were evaluated: a Runner task targeting level completion, a Hunter task rewarding enemy kills, and a Thief task rewarding coin collection. The fitness function went through several iterations; the choices behind the final version are described in Section 2.3.

2 Methodology

2.1 Environment

The Mario AI framework [4] is a Java simulator that exposes a socket API through which external agents send action vectors and receive observations at a fixed step rate. At each step the agent receives information about the simulation (e.g., Mario state & position, level scene observation grid).

The agent outputs a boolean vector [`LEFT`, `RIGHT`, `DOWN`, `JUMP`, `SPEED`] at each step, controlling Mario's actions. Difficulty is controlled along two independent axes: mario mode (small, large, or fire) and level difficulty, which scales obstacle density and enemy variety.

2.2 Typed Grammar and Representation

Each individual is a program tree in a typed grammar built on top of the provided DEAP [1] scaffold. We extended the original four types (`Cond`, `Key`, `Bool`, `Expr`) to nine: `Program`, `Stmt`, `Cond`, `Comparator`, `Key`, `Bool`, `Offset`, `EnemyKind`, and `TileValue`.

Program structure. A `Program` is a non-empty sequence of statements built left-recursively:

$$\text{Program} \rightarrow \texttt{END}(\text{Stmt}) \mid \texttt{SEQ}(\text{Stmt}, \text{Program})$$

Statements. A `Stmt` is a conditional branch or a direct key assignment:

$$\text{Stmt} \rightarrow \texttt{IF}(\text{Cond}, \text{Stmt}) \mid \texttt{IF\_ELSE}(\text{Cond}, \text{Stmt}, \text{Stmt}) \mid \texttt{SET\_ACTION}(\text{Key}, \text{Bool})$$

Conditions. Conditions combine two perception predicates with standard boolean connectives:

$$\begin{aligned}\text{Cond} \rightarrow\ & \texttt{AND}(\text{Cond}, \text{Cond}) \mid \texttt{OR}(\text{Cond}, \text{Cond}) \mid \texttt{NOT}(\text{Cond}) \\ & \mid \texttt{CheckEnemyAt}(\text{Offset}_x, \text{Offset}_y, \text{Comparator}, \text{EnemyKind}) \\ & \mid \texttt{CheckLandscapeAt}(\text{Offset}_x, \text{Offset}_y, \text{Comparator}, \text{TileValue}) \\ & \mid \texttt{IsMarioOnGround} \mid \texttt{MayMarioJump}\end{aligned}$$

Both predicates access the 22×22 grid at offset $(\Delta x, \Delta y)$ relative to Mario's cell. Horizontal offsets $\Delta x \in \{0, 1, 2, 3\}$ let the agent look ahead up to four cells; vertical offsets $\Delta y \in \{-3, \ldots, +3\}$ cover above and below. The rationale behind this is that giving it full grid visibility would lead an explosion of possible programs, while this small subset allows for sufficient behavioral coverage while allowing it to learn to react to close threats. Additionally, since the framework stores terrain and enemy data in the same grid, both predicates query the same array and differ only in the value range they compare against: enemy type codes (2 - 13) versus tile codes.

An earlier version of the grammar used precomputed boolean predicates computed outside the GP tree (`hole_ahead`, `enemy_ahead`, `wall_ahead`). This simplified the search space but also reduced it: the agent could not, for example, distinguish between enemy types or check arbitrary offsets. Though experiments with this grammar offered interesting insights (discussed below), these changes were reverted in order to align with the project statement's scope.

Terminals. Enemy kind terminals cover all sprite types: Goomba (standard and winged), Red and Green Koopa (standard and winged), Bullet Bill, Spiky (standard and winged), Piranha Flower, and Shell. Tile value terminals are HardObstacle (-10), Brick (16), and EnemyObstacle (20). Action key terminals are Right, Left, Jump, Speed, and Down.

2.3 Fitness Calculation

Three task variants were implemented: Runner, Hunter, and Thief. All three use the same per-step reward function; the difference is the episode-level bonus each adds to target its secondary objective.

Common Reward Every task uses the following per-step reward:

$$r_t = \Delta x_t + P_{\text{stuck}}(t) \tag{1}$$

Forward progress. $\Delta x_t = \mathbf{1}[\, x_t - x_{t-1} \geq 0.61 \,]$ awards one point when Mario advances at least 0.61 world units in a step. The binary form means walking and running at full speed score identically, since faster movement does not inherently produce better behavior given the grammar's limited action space. The threshold also prevents stationary oscillations near a wall from accumulating reward, which would work against the stuck penalty.

Stuck penalty. When Mario has made no forward progress for more than 10 consecutive steps and a solid obstacle occupies the cell directly ahead, a growing penalty is applied:

$$P_{\text{stuck}}(t) = \begin{cases} -10\,(n_{\text{stuck}} - 10) & \text{if } n_{\text{stuck}} > 10 \text{ and obstacle ahead} \\ 0 & \text{otherwise} \end{cases} \tag{2}$$

The 10-step grace period prevents the penalty from firing during legitimate pauses. The condition also requires that no Piranha Flower is present in the column above Mario, given that the original formulation without this check caused agents to rush into Piranha pipes instead of waiting for the enemy to retract.

Airtime penalty. A behavior that emerged early in training was a tendency to jump constantly. Given the grammar's limited expressive power, evolving a principled jump strategy is difficult; jumping on every possible frame clears a large fraction of obstacles and pits, which makes it a highly fit strategy in early generations. Once a population converges on it, there is no selection pressure toward a more targeted approach.

To counteract this, we tested a penalty proportional to total airtime across several iterations. The results were binary: either the penalty was too weak to suppress the behavior, or strong enough to also prevent agents from learning to jump at all, blocking forward progress entirely. This pointed to the conclusion that the current grammar does not provide enough perceptual structure for agents to learn when jumping is warranted. Additionally, agents evolved before removing the initial complex perceptions from our grammar (e.g., `hole_ahead`) did not exhibit constant jumping, which further supports this interpretation.

Runner Task The Runner task targets level completion. Beyond the progress reward, a win bonus of $W = 10{,}000$ is added at the end of any episode in which Mario clears the level:

$$B_{\text{runner}}(s, d) = W \cdot \mathbf{1}[\text{win}_{s,d}], \quad W = 10{,}000 \tag{3}$$

Hunter Task The Hunter task targets enemy kills. The episode bonus is proportional to the number of confirmed kills $\hat{K}_{s,d}$ accrued during the episode:

$$B_{\text{hunter}}(s, d) = K \cdot \hat{K}_{s,d}, \quad K = 10{,}000 \tag{4}$$

Kill detection compares the enemy list between consecutive frames, counting reductions in enemy count while handling the exceptions that occur with enemies from the first three difficulties: Koopa-to-Shell transitions (a Koopa count drop paired with a Shell count rise maintains the enemy list size, but is counted as a kill); enemies falling into pits (filtered by a y position threshold); enemies scrolling off-screen as Mario advances (filtered by horizontal distance); and Piranha Flowers retracting into pipes (detected by tracking per-column disappearances). A disappearing enemy is counted as a kill only if Mario's vertical velocity at that step is consistent with a stomp bounce, meaning a sign reversal or deceleration pattern matching a rebound off the top of an enemy.

This replaced an earlier heuristic that counted kills whenever the number of enemies within a small radius around Mario decreased between frames. That approach produced frequent false positives in crowded sections, inflating the reward signal and leading agents to stand still near enemy clusters rather than completing the level.

An earlier design combined kills and coin collection into a single secondary objective. Optimizing for both goals simultaneously, with the limited grammar

available to agents, added the workload of testing different weight configurations, and early experiments resulted in suboptimal agent performances across both tasks. Therefore, we created the Thief agent, splitting kills and coins into separate tasks, allowing each population to specialize on one behavior.

Thief Task The Thief task targets coin collection. Coin counts are read directly from the end-of-episode observation packet provided by the game framework, so no per-step detection is needed. The episode bonus is:

$$B_{\text{thief}}(s, d) = C \cdot \hat{N}_{s,d}, \quad C = 2{,}500 \tag{5}$$

where $\hat{N}_{s,d}$ is the total coins collected in the episode. The per-coin bonus (2,500) is set lower than the kill bonus (10,000) given that coins are more abundant than killable enemies in a typical level; a higher value would have dominated the forward-progress signal and incentivized agents to loop back and re-collect rather than advance.

Episode Fitness Each individual is evaluated over a seed pool $\mathcal{S}$ of 5 level seeds and 3 consecutive difficulty levels starting at base difficulty d_{base}. Fitness is the mean total reward across all $|\mathcal{S}| \times 3$ episodes:

$$F = \frac{1}{|\mathcal{S}|} \sum_{s \in \mathcal{S}} \sum_{d=d_{\text{base}}}^{d_{\text{base}}+2} \Big(\sum_{t} r_t + B_{\text{task}}(s, d) \Big) \tag{6}$$

where $B_{\text{task}}(s, d)$ is the task-specific episode bonus defined below.

2.4 Evolutionary Algorithm

Table 1 lists the final parameter configuration.

Selection and bloat control. We use Double Tournament selection [3], which runs two nested tournaments: the outer selects based on fitness (with a size that grows from 2 to 7 over training), and the inner selects based on tree size with a fractional tournament of size 1.7, preferring shorter programs when fitness is similar. This avoids adding a size term to the fitness function directly, which requires careful weighting.

Crossover and mutation. One-point crossover selects a random node in each parent and swaps the corresponding subtrees. If the resulting offspring exceeds height 10, the operation is reverted and the parent is preserved. Uniform subtree mutation selects a random node and replaces its subtree with a freshly grown random subtree (depth 2 - 6), so the algorithm introduces genuinely new code fragments rather than only recombining existing material.

Table 1. Evolutionary algorithm parameters used in the main experiments.

Parameter	**Value**
Population size	100
Generations	300
Crossover rate (p_c)	0.6
Mutation rate (p_m)	$1.0 \rightarrow 0.4$ (linear over training)
Crossover operator	One-point (`gp.cxOnePoint`)
Mutation operator	Uniform subtree (`gp.mutUniform`)
Selection	Double Tournament
Fitness tournament size	$2 \rightarrow 7$ (linear over training)
Parsimony tournament size	1.7
Max tree height	10
Elitism	Top-1 (Hall of Fame)

The mutation rate decreases from 1.0 to 0.4 over training. Preliminary experiments with fixed rates struggled with keeping an appropriate balance between exploration and exploitation at different points in the experiment. The dynamic schedule addresses this: high mutation early for exploration, lower mutation late for exploitation. The rationale behind the fitness tournament size is the same: it grows so that early generations explore under low selection pressure and later generations converge once the search has found promising regions.

Elitism. The single best individual across all generations, stored in a Hall of Fame (HoF), is copied directly into the next generation at each step. This ensures the best fitness found is never lost to a bad crossover or mutation.

Parallelism. The provided evaluation infrastructure distributes fitness evaluations across 5 worker processes, each holding a persistent connection to its own simulator instance. We extended it to support multi-seed pools, difficulty scheduling, and lazy re-evaluation: given the simulation's deterministic nature, only individuals whose fitness was invalidated need to be re-evaluated.

3 Experimental Design

3.1 Evaluation Protocol

All individuals within a generation are evaluated on the same 5 randomly drawn level seeds, so that fitness values are directly comparable within a generation. Using different seeds per individual, as we did in early experiments, introduced noise that made selection unreliable: an individual with higher fitness might simply have faced an easier level rather than exhibiting a better strategy, which led to the population stagnating on a few lucky seeds rather than improving.

To prevent overfitting to specific levels, the seed pool is rotated every 10 generations by drawing a fresh set from a universe of 1,000 seeds. At each rotation,

all population fitnesses are invalidated and re-evaluated under the new pool, including the HoF champion (so its score always reflects the current pool). A checkpoint is saved at each rotation. At the end of training, all checkpoints are re-evaluated on a fixed set of 500 held-out seeds across the three training difficulties. The HoF champion from the checkpoint with the highest overall win rate on this held-out set is selected as the final evolved agent. This step is necessary because the HoF score stored in each checkpoint reflects fitness under whichever seed pool was active during that rotation, making raw fitness values across checkpoints incomparable without a shared evaluation basis. This method also allows for the visualization of performance across generations, as seen in Figure 1.

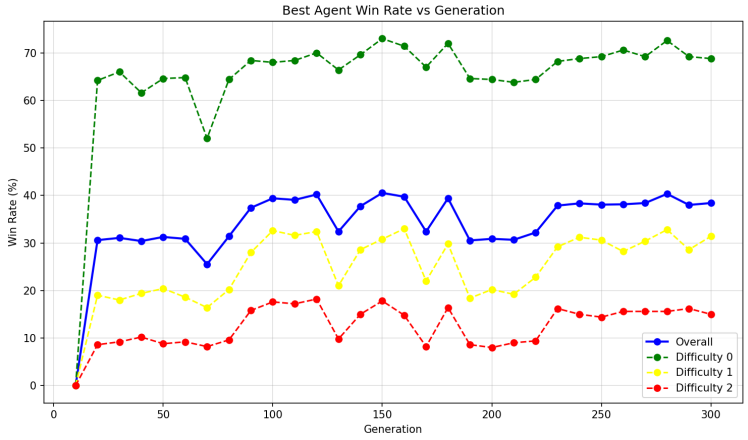

Fig. 1. Example champion win rate vs. generation plot for the Runner task. After reaching finding a capable strategy (running and jumping), it hovers around 30-40% win rate, attempting to optimize its strategy until the end of the training run. Completion rate per level closely follows the swings in the overall rate.

For seed pool rotation, periods of 5, 10, and 20 generations were compared, with the middle option achieving overall better results. Rotating every 5 generations did not give the population enough time to exploit each seed pool before it changed, slowing convergence. Rotating every 20 generations allowed the population to overfit to specific level seeds, with agents that scored well on the current pool but failed to generalize when the pool changed. Rotating every 10 generations provided a good balance: enough stability for selection to act meaningfully, and enough change to prevent overfitting.

The base difficulty d_{base} is fixed at 0 throughout training, so agents are always evaluated on difficulties 0, 1, and 2 simultaneously. An earlier design explored a curriculum that gradually shifted d_{base} upward over generations. This was abandoned when training on all three difficulties from the start produced better-generalizing agents, likely because the easier difficulty levels provide a reliable learning signal even when the population is still immature.

Finally, we chose to run 300-generation runs as a compromise between computational cost and exploration of the evolutionary approach. 200 and 100-individual populations were compared, with the latter configuration producing comparable completion rates at half the cost, and was adopted as the default.

3.2 Comparative Baseline & Statistical Analysis

Evolved champions are compared against a random search baseline to separate the contribution of the evolutionary process from the expressive power of the grammar alone. We run five independent experiments per task variant, each with a different random seed. For each run, the champion is selected from checkpoints using the procedure described above, with win rate as the selection criterion for Runner, average kills for Hunter, and average coins for Thief. The baseline for each run is a Champion obtained by evaluating HoF agents from checkpoints of 100 random individuals and choosing the best-performing one. Both agents are then evaluated on 5,000 held-out level seeds across three difficulty levels.

The evolutionary process and level generator are both stochastic, so a single run per task is not sufficient to draw reliable conclusions. Across the five independent runs per variant, we report mean and standard deviation for completion rate, average distance in unbeaten levels, enemies killed, and coins collected.

4 Results and Analysis

Table 2 compares evolved champions against the random search baseline across all three task variants, evaluated on 5,000 held-out seeds $\times$ 3 difficulties. Table 3 breaks down win rate by difficulty, showing non-conditional (NC) and conditional (C) rates side by side.

Table 2. Evolved champions vs. random search baseline across task variants (5,000 seeds $\times$ 3 difficulties, mean $\pm$ std across runs).

Metric	Random Baseline	Runner	Hunter	Thief
Levels Cleared (%)	0.3 ± 0.5	$\mathbf{39.3 \pm 0.2}$	32.2 ± 0.6	30.5 ± 0.5
Max Distance (px)	458.9 ± 53.4	$\mathbf{1005.3 \pm 2.5}$	942.3 ± 10.6	928.9 ± 17.8
Enemies Killed	0.07 ± 0.16	0.63 ± 0.06	$\mathbf{0.88 \pm 0.04}$	0.83 ± 0.03
Coins Collected	1.03 ± 1.79	6.99 ± 0.03	$\mathbf{9.69 \pm 0.33}$	9.02 ± 1.00

Table 3. Win rate (%) per difficulty level (mean $\pm$ std across runs), shown as NC / C (non-conditional / conditional). The conditional rate at $d > 0$ is computed over the subset of episodes in which the agent also won at difficulty $d - 1$.

Diff.	Runner (NC / C)	Hunter (NC / C)	Thief (NC / C)
($d = 0$)	$\mathbf{69.9 \pm 0.9}$	64.0 ± 0.2	62.5 ± 1.2
($d = 1$)	$\mathbf{31.8 \pm 0.1}$ / $\mathbf{33.0 \pm 0.4}$	23.0 ± 1.2 / 23.9 ± 1.4	20.6 ± 0.7 / 21.8 ± 1.4
($d = 2$)	$\mathbf{16.3 \pm 1.5}$ / $\mathbf{14.6 \pm 1.0}$	9.5 ± 1.4 / 12.4 ± 2.2	8.3 ± 0.3 / 8.9 ± 2.4

All three evolved agents outperform the random baseline in all metrics, given that it struggles to complete any level and usually winds up dying early or getting stuck on a wall due to not learning to jump. It also displays high standard deviations across all metrics, as it does not obtain a solidified strategy across individuals, unlike the other tasked agents, which consistently leans toward the variation of the "run-jump" strategy that is better suited for its task. The fact that Runner has a considerably higher win rate than Hunter and Thief aligns with the expected behavior: while all incentivize forward progress, Runner is the one that directly rewards beating levels. However, Hunter and Thief don't fall behind too sharply: their secondary goals appear to serve as an effective proxy for winning levels (the further they reach in the level, the better their chances of achieving higher scores).

For kills and coins, Hunter and Thief outperform Runner in both metrics. We theorize that, due to individuals of the former two tasks tendentially being slower (using the SPEED control less), they cover more space and have more opportunities to interact with enemies and coins (thus also dying more frequently to enemy non-kills and pits). Between the two, Hunter is the most effective at utilizing this strategy, even collecting more coins than Thief. This may also be explained by the fact that there is no mapping for coins in the perception grid, essentially meaning it is much harder to optimize for this goal.

Table 3 shows a consistent difficulty spike across all tasks: win rates fall from around 65% at $d = 0$ to 25% at $d = 1$ and 12% at $d = 2$. In most cases, the conditional rate exceeds the non-conditional rate, meaning that beating a medium-difficulty level is predictive of beating a hard one. However, for Runner, the conditional rate at $d = 2$ is 14.6% against a non-conditional 16.3%, a result that is statistically significant ($p < 0.05$). This may indicate that the additional challenges introduced by $d = 2$ compared to $d = 1$ may require a slightly different skill set to beat, and that beating $d = 1$ is not a reliable predictor of beating $d = 2$ when using this strategy.

Table 4. Pairwise comparisons of key metrics across evolved agents. Covers mean difference Δ, 95% confidence interval for Δ, Cohen's d, and result of a two-tailed Welch's t-test $n = 5$

Comparison	Δ	**CI**	**Cohen's** d	t	**df**	p
Runner vs. Random (levels cleared, %)	39.04	[38.5, 39.6]	112.1	177.3	5.5	< 0.0001
Hunter vs. Runner (enemies killed)	0.25	[0.19, 0.31]	4.90	7.75	7.0	< 0.0001
Hunter vs. Thief (enemies killed)	0.05	[0.01, 0.09]	0.42	2.24	1.41	0.028
Hunter vs. Thief (coins collected)	0.67	[-0.54, 1.88]	0.90	1.42	4.9	0.23

In Table 4 we make several pairwise comparisons between task types to isolate task-specific gains. The first two comparisons confirm extremely large advantages. The last two show that while Hunter's advantage over Thief on killed enemies is statistically significant, the advantage on coins is not.

5 Conclusion

GP over a typed grammar produced controllers that clear levels at 39.3% across 5,000 held-out seeds, against 0.3% for random search. Hunter and Thief agents reached 32.2% and 30.5% respectively; Hunter led on kills (0.88 vs. 0.63 for Runner) and incidentally on coins, though the coin advantage over Thief is not statistically reliable ($p = 0.23$). All three converge on "run-jump" variants, with the secondary objective shaping which variant is preferred. The biggest gains came from a small number of decisions: splitting the combined secondary objective into separate tasks, switching to a type-aware kill detector that eliminated false positives from the radius heuristic, and rotating the seed pool every 10 generations to prevent level overfitting without disrupting selection.

The grammar's fixed arity and absence of arithmetic or loops is the main structural limitation: the airtime penalty experiment showed agents cannot evolve a conditional jump policy without richer perceptual primitives.

References

1. Fortin, F.A., De Rainville, F.M., Gardner, M.A., Parizeau, M., Gagné, C.: DEAP: Evolutionary algorithms made easy. Journal of Machine Learning Research **13**, 2171–2175 (2012)
2. Koza, J.R.: Genetic Programming: On the Programming of Computers by Means of Natural Selection. MIT Press, Cambridge, MA (1992)
3. Luke, S., Panait, L.: A comparison of bloat control methods for genetic programming. Evolutionary Computation **14**(3), 309–344 (2006). https://doi.org/10.1162/evco.2006.14.3.309
4. Togelius, J., Karakovskiy, S., Koutník, J., Schmidhuber, J.: Super Mario evolution. In: Proceedings of the IEEE Symposium on Computational Intelligence and Games (CIG). pp. 156–161. IEEE (2009). https://doi.org/10.1109/CIG.2009.5286481